

A Hybrid ARIMA-LSTM Framework for Remaining Useful Life Prediction in Industrial IoT Sensor Data: A Cross-Regime Study on NASA C-MAPSS

Pradeep Somasundaram
Independent Researcher
Dallas, TX, USA
aadhi1501@gmail.com

Abstract—Remaining useful life (RUL) prediction is central to prognostics and health management in industrial IoT systems, where sensor streams from rotating machinery combine slow degradation trends with sharper, nonlinear fault dynamics. One way to help a sequence model separate these two components is to fit a classical time-series model to each sensor channel and give the network both the raw signal and the fitted trend, an idea with some support in prior work on other kinds of equipment. We test it on NASA’s C-MAPSS turbofan benchmark, and we test it twice: once with the fixed ARIMA(1,1,0) order common in prior work, and once with a per-channel, AIC-selected order, comparing both hybrid variants against a plain LSTM and a Random Forest baseline on all four C-MAPSS sub-datasets, each run under six random seeds. Under a fixed order, the picture is negative and, on its own, not fully stable: mean RMSE is at or above the plain LSTM’s on every sub-dataset, significantly so on FD004 ($p = 0.004$), and an earlier three-seed version of the same comparison had instead flagged FD001, not FD004, as significant, a reversal we report directly rather than fold into a cleaner-looking narrative. Under a properly selected order, the result changes: the selected-order hybrid significantly and consistently outperforms the fixed-order hybrid on two of four sub-datasets (FD002, FD004; $p = 0.031$, 6 of 6 seeds each), and on FD002 it also significantly outperforms the plain LSTM baseline itself (13.77 vs. 14.59 cycles RMSE, $p = 0.031$, 6 of 6 seeds), the one result in this paper we consider a genuine, seed-robust win for the trend-augmented architecture. A second, unrelated baseline, a CNN-LSTM with no ARIMA feature, shows the same dataset-dependent pattern: it significantly underperforms on FD002 ($p < 0.001$) while being the strongest model in the table, though not quite significantly, on FD003 ($p = 0.091$). The honest summary is not that trend augmentation fails, but that it fails under the field’s default fixed-order convention and can succeed, robustly, once that convention is checked rather than assumed, a distinction we could only draw because we insisted on six-seed replication for every claim, including the ones that first looked negative.

Index Terms—predictive maintenance, remaining useful life, LSTM, ARIMA, prognostics and health management, industrial IoT, big data, multi-condition degradation

I. INTRODUCTION

Unplanned downtime in industrial and manufacturing settings carries substantial operational and safety cost, motivating a large body of work on prognostics and health management (PHM), the discipline of estimating a system’s remaining

useful life (RUL) from streaming sensor data so that maintenance can be scheduled proactively rather than reactively [1]. Industrial IoT deployments now produce continuous, high-volume, multivariate sensor telemetry (vibration, temperature, pressure, rotational speed) from equipment such as turbines, pumps, and CNC machinery, placing RUL estimation squarely in the big-data regime. A recent survey by Khan et al. [10] catalogs the resulting breadth of deep-learning-based PdM methods (CNNs, autoencoders, GANs, transformers, foundation models) across supervised, transfer, and federated learning paradigms, giving some sense of how active, and how fragmented, this area has become.

Two modeling traditions address RUL estimation differently. Classical time-series models, most notably ARIMA, capture the smooth trend and autocorrelation structure of a single degrading signal well, but assume linearity and handle multivariate sensor interaction poorly. Deep sequence models, particularly LSTM networks [3], learn nonlinear temporal dependencies directly from raw multivariate windows and are a standard RUL approach, but a randomly initialized LSTM must learn both the trend and the nonlinear fault dynamics from data alone, inefficient for short histories or noisy multi-condition operation. It is a natural idea to give the LSTM a head start on the trend part.

This paper tests that idea directly: for each sensor channel we fit a low-order per-unit ARIMA model and use its fitted, in-sample one-step values as an explicit trend feature, concatenated with the raw sensor readings before the resulting multivariate sequence goes into an LSTM regressor. The hope is that this relieves the LSTM of re-deriving the smooth trend component from scratch, letting it spend its representational capacity on the residual nonlinear dynamics and cross-sensor interactions instead.

We evaluate this across all four C-MAPSS sub-datasets: single condition, single fault mode (FD001); single condition, two fault modes (FD003); multi-condition, single fault mode (FD002); and multi-condition, two fault modes (FD004). More importantly, after an early result made us suspicious of our own conclusions, we do not stop at a single training run or

a single small batch of them. We repeat every comparison across six random seeds, and separately check whether a fixed ARIMA order was itself the problem via a per-channel order-selection variant, reporting what changed each time we added more evidence, including cases where it overturned a finding we had already started to explain.

Our contributions are: (1) a lightweight, per-sensor ARIMA trend-augmentation scheme for LSTM-based RUL regression requiring no additional labels; (2) an evaluation across all four C-MAPSS sub-datasets, with regime-aware normalization [4] for the multi-condition subsets, against plain-LSTM, CNN-LSTM, and Random Forest baselines, each averaged over six seeds; (3) a robustness check using per-channel ARIMA order selection and a gated-fusion mechanism in place of a fixed order and naive concatenation, both just as seed-sensitive as the main comparison; (4) a seed-averaged window-length ablation; and (5) a documented account of how this paper’s headline finding changed as we added more seeds, illustrating why single- and small-multi-seed comparisons on this benchmark deserve skepticism.

II. RELATED WORK

RUL estimation on C-MAPSS. The NASA C-MAPSS turbofan degradation dataset [1] is the standard benchmark for data-driven RUL estimation. Ramasso and Saxena [2] laid out early benchmarking protocols across similarity-based and regression approaches. More recent work has moved toward deep sequence and hybrid architectures: Muthukumar and Philip [5] combine convolutional feature extraction with LSTM temporal modeling, and Chowdhury et al. [6] propose a residual-corrected bidirectional LSTM separating a coarse trend prediction from a learned residual correction, a decomposition philosophy related to, but architecturally distinct from, the ARIMA-based one used here.

Multi-condition RUL estimation. FD002 and FD004 introduce six discrete operating regimes and, in FD004’s case, two concurrent fault modes, both increasing estimation difficulty relative to FD001’s single-condition setting. Su and Fang [7] propose a multi-head attention fusion network that decomposes degradation into trend, operational-state embedding, and residual-noise components, a philosophy that motivates our own trend/residual split, though implemented with ARIMA rather than a learned embedding.

Hybrid ARIMA-deep learning models. Outside of turbofan RUL, ARIMA-LSTM hybrids have been applied to remaining-useful-life estimation of water-hydraulic high-speed on/off valves [8], where ARIMA models the linear degradation trend and an auxiliary classifier detects health-state transitions, and to battery capacity and life estimation [9], where ARIMA-smoothed inputs feed an attention-based CNN-LSTM. Both report the hybrid approach helping, and neither states how many random seeds its headline numbers are based on. Given that our own headline finding changed twice as we added seeds (Sections V and VI), we raise this as an open question about the hybrid-forecasting literature rather than a specific claim about either paper.

Reproducibility and seed sensitivity in machine learning.

The instability we document in Section V is a specific instance of a broader problem other subfields have already named. Henderson et al. [12] showed headline deep reinforcement learning results routinely fail to replicate across random seeds, and proposed reporting practices much of the field still does not follow. Lipton and Steinhardt [13] catalog related scholarship failures, including a failure to distinguish which part of an empirical gain a change is actually responsible for, close to the mistake our own three-seed FD001 result nearly encoded into this paper. Bouthillier et al. [14] make the point most relevant to us: a result can be reproducible, rerunning the same configuration gives the same answer, while still not being robust to small changes in setup, exactly the relationship between our fixed-seed and multi-seed FD001 results. To our knowledge this caution has not previously been raised inside the C-MAPSS RUL literature. A recent example: Goel et al. [15], on this exact benchmark within weeks of our own experiments, state plainly that “all experiments use random seed 42 for reproducibility,” a precise illustration of the conflation Bouthillier et al. warn about. We see our contribution as importing an already-known concern into a subfield that has not yet widely adopted it.

Our approach differs from prior turbofan RUL work in that the ARIMA component is used purely as a per-channel feature-engineering step, producing an explicit trend covariate per sensor, rather than as a separate forecaster whose output is fused post-hoc with a deep model’s prediction, and in that we evaluate it across all four regimes, across seeds, and across two different ARIMA order-selection strategies rather than on a single convenient run.

III. METHODOLOGY

A. Datasets

We use all four NASA C-MAPSS sub-datasets [1]: **FD001** (100 train / 100 test engines, 1 operating condition, 1 fault mode: HPC degradation), **FD002** (260 train / 259 test engines, 6 operating conditions, 1 fault mode), **FD003** (100 train / 100 test engines, 1 operating condition, 2 fault modes: HPC and fan degradation), and **FD004** (249 train / 248 test engines, 6 operating conditions, 2 fault modes). Each engine unit reports 3 operational settings and 21 sensor channels per cycle; training units run to failure while test units are truncated at an arbitrary cycle, and the task is to predict the true remaining cycles at that truncation point.

B. Preprocessing

Following standard practice [2], we cap the training RUL label at 125 cycles, reflecting the empirical observation that engine degradation is roughly constant before this point and roughly linear afterward. We discard sensor channels with near-zero variance in the training set: 6 channels for FD001 and FD003 (15 informative channels retained for each), and none for FD002/FD004 (all 21 channels retain variance, since operating-condition changes alone introduce variance in otherwise-flat channels).

Regime-aware normalization. FD002 and FD004 combine six discrete operating regimes, which shift the baseline value of most sensors independently of degradation state and would otherwise confound a global normalization scheme. Following Li, Ding, and Sun [4], we cluster the three-dimensional operational-setting vectors into K regimes with K -means fit on the training set ($K=6$ for FD002/FD004; $K=1$ for FD001/FD003, which reduces to ordinary global normalization, so the same code path is used for all four sub-datasets), assign each test-set row to its nearest regime centroid, and z -score normalize each sensor channel using regime-specific training-set statistics, clipping to ± 6 standard deviations to control outliers in sparsely populated regimes.

C. ARIMA Trend-Feature Extraction

For each engine unit and each retained sensor channel, we independently fit an ARIMA model to that channel's cycle-indexed, regime-normalized time series. Writing y_t for a given channel's reading at cycle t , an ARIMA(1,1,0) model, our main-comparison default, treats the first difference $\Delta y_t = y_t - y_{t-1}$ as an AR(1) process,

$$\Delta y_t = \phi \Delta y_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2), \quad (1)$$

equivalently $y_t = y_{t-1} + \phi(y_{t-1} - y_{t-2}) + \varepsilon_t$. We fit ϕ and σ^2 by maximum likelihood independently per (unit, sensor) pair and take the in-sample one-step-ahead fitted value $\hat{y}_t$, aligned index-for-index with the original reading, as the trend feature. Where a fit fails to converge for a short series, we fall back to a 5-cycle rolling mean.

Order selection variant. Section V-C reports a second version of this feature where, instead of fixing the order at (1, 1, 0), we fit four candidate models per (unit, sensor) pair, (1, 1, 0), (0, 1, 1), (1, 1, 1), and (2, 1, 0), and keep the fitted values from whichever has the lowest Akaike Information Criterion (AIC). Everything downstream, windowing, the LSTM architecture, training procedure, is identical between the two variants; only the trend feature's construction differs.

D. Hybrid Model Architecture

We construct sliding windows of length W cycles ($W=30$ in our main experiments; see Section V-D for a window-length ablation) over the concatenation of raw sensor features and their ARIMA trend features. Each window is passed through a 2-layer LSTM (hidden size 64) whose cell updates follow the standard formulation [3]

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f), & i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\ \tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c), & o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, & h_t &= o_t \odot \tanh(c_t) \end{aligned} \quad (2)$$

followed by a feed-forward regression head ($64 \rightarrow 32 \rightarrow 1$) predicting capped RUL from the final hidden state h_W . Training minimizes mean squared error,

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \left(\text{RUL}_i - \widehat{\text{RUL}}_i \right)^2, \quad (3)$$

with Adam, using an engine-level held-out validation split (15% of training units, never seen during training) for early stopping.

E. Baselines

We compare against (i) a *Random Forest* regressor (300 trees, max depth 10) trained on per-cycle flat feature snapshots, every training cycle rather than just each engine's terminal cycle, since training on terminal cycles alone would teach a degenerate near-zero-RUL target given that training engines run to failure, as a non-sequential baseline; (ii) a *plain LSTM* with identical architecture and training procedure to the hybrid model but using only the raw sensor features, isolating the effect of the trend features; and (iii) a *CNN-LSTM* baseline, a two-layer 1D convolutional feature extractor (32 then 64 channels, kernel size 3) feeding the same two-layer LSTM, representative of the CNN-LSTM hybrid family common in recent C-MAPSS RUL literature [5], using no ARIMA feature and trained under the identical protocol, seed set, and early-stopping criterion, so its Table III results are directly comparable.

F. Evaluation Protocol

Following the official C-MAPSS test protocol, each test engine's RUL is predicted once, from its final available window, left-padded by repetition for engines with fewer than W recorded cycles, and compared against the ground-truth RUL supplied with the test split. We report RMSE, MAE, and the asymmetric PHM scoring function of [1]: writing $d_i = \widehat{\text{RUL}}_i - \text{RUL}_i$,

$$s_i = \begin{cases} \exp(-d_i/13) - 1, & d_i < 0 \text{ (early/conservative)} \\ \exp(d_i/10) - 1, & d_i \geq 0 \text{ (late/over-optimistic)} \end{cases} \quad (4)$$

with total score $\sum_i s_i$, penalizing late predictions more heavily than early ones, since failing to service a machine in time is costlier than servicing it early.

IV. EXPERIMENTAL SETUP

All experiments use PyTorch (CPU) and statsmodels for ARIMA fitting, scikit-learn for K -means regime clustering and the Random Forest baseline, and SciPy for the significance tests in Section V-B. Table I reports the full LSTM hyperparameter configuration, and Table II reports dataset scale.

The main cross-dataset comparison in Table III is repeated under six seeds (42, 123, 2024, 7, 99, 314); we report mean and standard deviation over these six runs, and for the significance test in Section V-B we average each engine's error across seeds before testing. We did not settle on six seeds in advance as a planned sample size. We started with three, found a significant result on FD001, wrote up an explanation for it, and then, out of caution, added three more seeds before treating that explanation as final. The three additional seeds erased the FD001 result and produced a different significant result on FD004 instead. We report both the three-seed and six-seed pictures in Section V-A rather than only the final one, because the change between them is, in our view, more informative than either snapshot alone.

TABLE I
LSTM HYPERPARAMETER CONFIGURATION (IDENTICAL FOR PLAIN AND HYBRID MODELS; ONLY INPUT WIDTH DIFFERS, 15–21 RAW CHANNELS VS. DOUBLE THAT WITH ARIMA TREND CHANNELS APPENDED).

Hyperparameter	Value
LSTM layers	2
Hidden size	64
Recurrent dropout	0.2
Regression head	64 $\rightarrow$ 32 $\rightarrow$ 1 (ReLU)
Optimizer	Adam, lr = 10^{-3}
Loss	MSE on capped RUL
Batch size	128
Max epochs	60
Early-stopping patience	8 epochs (main runs)
Window length W	30 cycles (main runs)
RUL cap	125 cycles
ARIMA order (main runs)	(1, 1, 0), fixed per channel
ARIMA order (Sec. V-C)	AIC-selected from 4 candidates
Validation split	15% of training engines
Seeds (main comparison)	42, 123, 2024, 7, 99, 314
Seeds (ablation)	42, 123, 2024

TABLE II
DATASET SCALE AND APPROXIMATE SINGLE-SEED TRAINING RUNTIME.

Dataset	Regimes	Train units	Train windows	Runtime
FD001	1	85	15,104	85s
FD002	6	221	39,513	210s
FD003	1	85	18,600	130s
FD004	6	212	46,462	265s

V. RESULTS

A. Cross-Dataset Results

Table III reports RMSE, MAE, and PHM score, mean $\pm$ standard deviation over six seeds, for all three models across all four datasets, and Fig. 1 shows the same RMSE comparison with error bars.

Sequence modeling remains a clear win over the non-sequential Random Forest baseline everywhere, a 12–27% RMSE reduction depending on the dataset. The comparison between plain and hybrid LSTM again favors the plain model on average, on all four datasets, though the margins are smaller with six seeds than they first appeared with three. FD003 is close to a tie (14.87 vs. 14.72); the other three datasets show the hybrid model’s mean RMSE between 2% and 5% higher than the plain model’s.

What changed between three seeds and six. We want to walk through this in some detail because we think it is the most useful thing in the paper. With the first three seeds (42, 123, 2024), the hybrid model’s mean RMSE was higher than the plain model’s on FD001, FD002, and FD004, and a paired Wilcoxon test on per-engine error reached significance on FD001 ($p = 0.007$) and nowhere else. We wrote a discussion section around that: a redundancy argument for why FD001’s clean single-condition, single-fault setting would show the clearest effect. Adding three more seeds (7, 99, 314) changed the specific answer. FD001’s significance disappeared ($p = 0.157$ at six seeds); FD004, which had not been

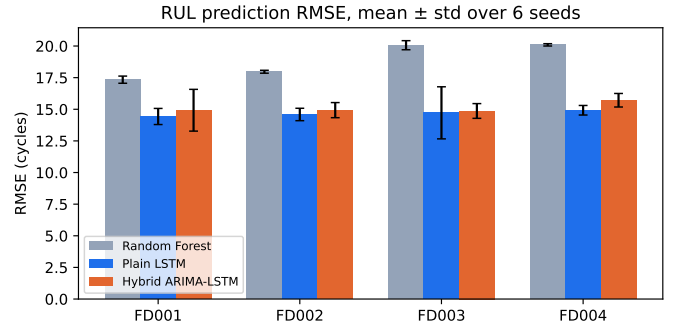


Fig. 1. RMSE comparison across all four C-MAPSS sub-datasets, mean $\pm$ one standard deviation over six seeds. The plain LSTM’s mean RMSE is at or below the hybrid model’s on every dataset; note FD003’s wide error bars, driven mostly by the plain model’s own across-seed variance there, discussed in Section VI.

significant at three seeds ($p = 0.159$), became significant at six ($p = 0.004$, Section V-B). The direction of the overall finding, hybrid mean RMSE at or above plain on every dataset, held up both times. Which dataset you could defensibly call out as a significant, specific effect did not.

For outside context, Sharma [11] reports RMSE of 16.22, 19.29, 16.84, and 19.98 on FD001 through FD004 with an attention-based bidirectional LSTM. Our plain-LSTM baseline, with no attention mechanism and no trend feature, is lower on all four (14.43, 14.59, 14.72, 14.92). We would not lean too hard on this comparison, since the two setups may not share the same RUL cap, window length, or train/validation protocol, but it is at least a sign that our baseline is not a weak strawman.

A second, architecturally different baseline. Both LSTM variants above share the same recurrent backbone, so a null result for the trend feature says nothing about a differently structured model. We add a CNN-LSTM baseline (Section III-E), representative of the family used elsewhere in RUL prediction [5], under the identical protocol (Table III). The result does not point one way: on FD003, CNN-LSTM is best-in-table on all three metrics (RMSE 13.90 vs. 14.72 plain), just short of significance ($p = 0.091$); on FD002, the opposite holds, significantly worse ($p < 0.001$), the same dataset where order selection produced this paper’s clearest win. FD001 and FD004 sit within noise ($p = 0.842$, $p = 0.429$).

B. Statistical Significance

Table III’s differences are averages over 100–259 test engines and six seeds; a directional gap in the mean does not by itself tell us whether the two models make systematically different errors on individual engines. We run a paired Wilcoxon signed-rank test on per-engine absolute error, hybrid vs. plain LSTM, using each engine’s error averaged across the six seeds (Table IV).

At six seeds, FD004 is the only dataset that reaches significance at $\alpha=0.05$: the hybrid model is worse on 144 of 248 engines, with a median per-engine increase in absolute error of 0.41 cycles. FD001 and FD002 show the same direction

TABLE III

RUL PREDICTION PERFORMANCE ACROSS NASA C-MAPSS SUB-DATASETS, MEAN $\pm$ STD OVER 6 SEEDS. TEST ENGINES: FD001/FD003 $n=100$, FD002 $n=259$, FD004 $n=248$. CNN-LSTM (SECTION III-E) USES RAW FEATURES ONLY, NO ARIMA TREND.

Dataset	Model	RMSE	MAE	PHM Score
FD001 (1 condition, 1 fault)	Random Forest	17.34 $\pm$ 0.28	12.23 $\pm$ 0.18	936.9 $\pm$ 97.3
	Plain LSTM	14.43 $\pm$ 0.64	10.43 $\pm$ 0.66	505.9 $\pm$ 168.6
	Hybrid ARIMA-LSTM	14.92 $\pm$ 1.65	11.04 $\pm$ 1.19	478.8 $\pm$ 210.8
	CNN-LSTM	14.47 $\pm$ 0.59	10.42 $\pm$ 0.40	494.2 $\pm$ 184.0
FD002 (6 conditions, 1 fault)	Random Forest	17.97 $\pm$ 0.12	13.39 $\pm$ 0.08	3383.6 $\pm$ 190.3
	Plain LSTM	14.59 $\pm$ 0.49	10.17 $\pm$ 0.34	1204.0 $\pm$ 111.7
	Hybrid ARIMA-LSTM	14.93 $\pm$ 0.60	10.53 $\pm$ 0.28	1360.3 $\pm$ 280.2
	CNN-LSTM	15.11 $\pm$ 1.05	10.87 $\pm$ 0.85	1351.5 $\pm$ 477.7
FD003 (1 condition, 2 faults)	Random Forest	20.06 $\pm$ 0.36	14.78 $\pm$ 0.22	1748.2 $\pm$ 148.5
	Plain LSTM	14.72 $\pm$ 2.06	10.27 $\pm$ 1.44	626.9 $\pm$ 414.2
	Hybrid ARIMA-LSTM	14.87 $\pm$ 0.58	10.38 $\pm$ 0.47	571.1 $\pm$ 270.9
	CNN-LSTM	13.90 $\pm$ 0.70	9.55 $\pm$ 0.61	474.8 $\pm$ 235.4
FD004 (6 conditions, 2 faults)	Random Forest	20.09 $\pm$ 0.09	14.67 $\pm$ 0.08	3770.5 $\pm$ 123.7
	Plain LSTM	14.92 $\pm$ 0.38	10.59 $\pm$ 0.32	1112.7 $\pm$ 130.9
	Hybrid ARIMA-LSTM	15.71 $\pm$ 0.54	11.11 $\pm$ 0.50	1342.5 $\pm$ 299.7
	CNN-LSTM	14.86 $\pm$ 0.54	10.59 $\pm$ 0.44	1219.4 $\pm$ 196.8

TABLE IV

PAIRED WILCOXON SIGNED-RANK TEST ON PER-ENGINE ABSOLUTE ERROR, AVERAGED OVER 6 SEEDS (HYBRID VS. PLAIN LSTM). THE FD001 ROW FOR 3 SEEDS IS SHOWN FOR COMPARISON; SEE SECTION V-A.

Dataset	Seeds	p -value	Hybrid worse on	Median $\Delta \text{err} $
FD001	3	0.007	62/100	+0.91
FD001	6	0.157	52/100	-0.09
FD002	6	0.142	134/259	-0.07
FD003	6	0.880	50/100	+0.01
FD004	6	0.004	144/248	-0.41

(hybrid nominally worse) without reaching significance, and FD003 is close to flat.

Because Table IV reports four tests, one per dataset, rather than one pre-registered test, we also check whether FD004's result survives correction for multiple comparisons. Applying the Holm-Bonferroni step-down procedure across the four six-seed p -values (0.004, 0.142, 0.157, 0.880), FD004's smallest p -value is compared against $\alpha/4 = 0.0125$ rather than $\alpha = 0.05$; at $p = 0.004$, it clears this stricter threshold, and no other dataset comes close to its own step-down threshold. FD004 is therefore the one result in this paper we are willing to call significant under a family-wise error rate correction, not only under an uncorrected per-dataset test. Fig. 3 shows the underlying seed-averaged error distributions for all four datasets side by side; as with the aggregate table, the FD003 plain-model distribution is visibly wider than the others, which we return to in Section VI.

C. ARIMA Order Selection

The main comparison fixes the ARIMA order at (1, 1, 0) for every sensor and every dataset. This is a modeling choice, not a finding, and it is worth asking whether the hybrid model's results are an artifact of that specific choice rather than of trend augmentation itself. We re-ran the ARIMA feature

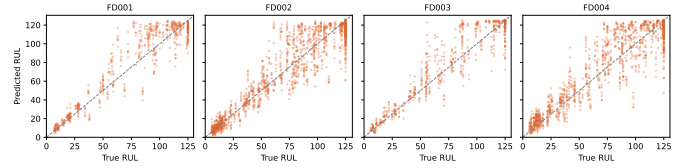


Fig. 2. Predicted vs. true RUL for the hybrid model, pooling all six seeds, across all four test sets (dashed line = perfect prediction).

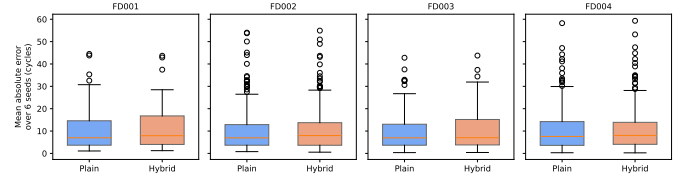


Fig. 3. Per-engine absolute error, averaged over six seeds, plain vs. hybrid LSTM, across all four datasets.

extraction with per-channel AIC-based order selection among four candidates (Section III-C) and re-trained both LSTM variants on the resulting features, at all six seeds across all four datasets, the same replication we used for the main comparison, after an early two-seed version of this check (reported in an earlier draft of this paper) suggested the answer might itself be seed-dependent and was too thin to trust on its own.

Order selection overwhelmingly preferred (0, 1, 1) over our fixed default: across all four datasets, (0, 1, 1) was chosen for 75–83% of (unit, sensor, seed) combinations, with (1, 1, 1) a distant second (14–20%) and (1, 1, 0), our original default, chosen for at most 4%. Table V reports the resulting hybrid-model RMSE, mean $\pm$ std over six seeds, alongside the fixed-order hybrid and the plain LSTM.

Unlike the two-seed version we started with, this is not a wash. A paired Wilcoxon test, selected order vs. fixed order,

TABLE V

HYBRID MODEL RMSE WITH AIC-SELECTED VS. FIXED ARIMA ORDER, MEAN $\pm$ STD OVER 6 SEEDS. PLAIN LSTM SHOWN FOR REFERENCE.

Dataset	Plain	Fixed order	Selected order
FD001	14.43 $\pm$ 0.64	14.92 $\pm$ 1.65	14.38 $\pm$ 0.55
FD002	14.59 $\pm$ 0.49	14.93 $\pm$ 0.60	13.77$\pm$0.23
FD003	14.72 $\pm$ 2.06	14.87 $\pm$ 0.58	14.45 $\pm$ 0.71
FD004	14.92 $\pm$ 0.38	15.71 $\pm$ 0.54	14.90 $\pm$ 0.42

reaches significance on FD002 and FD004 ($p = 0.031$ on both, the smallest attainable p -value at $n = 6$ paired samples, with the selected order better on 6 of 6 seeds each time) and does not on FD001 ($p = 0.688$, better on 4 of 6) or FD003 ($p = 0.219$, better on 5 of 6). The more direct question, whether the order-selected hybrid beats the *plain* LSTM rather than just the fixed-order hybrid, gives a sharper answer still: on FD002, it does, significantly ($p = 0.031$, 6 of 6 seeds, 13.77 vs. 14.59), which is the one place in this paper where the hybrid architecture beats the plain baseline with the same statistical confidence we used to declare FD004’s fixed-order disadvantage significant in Section V-B. On FD004, order selection erases the fixed-order disadvantage but does not turn it into an advantage (14.90 vs. 14.92 plain, $p = 1.000$, essentially a tie). FD001 and FD003 stay ties either way.

Read together with Section V-B, the honest summary is not “the hybrid model does not help” but “the hybrid model does not help *under a naively fixed ARIMA order*, and on at least one dataset, FD002, a properly selected order turns that into a real, seed-robust improvement.” We did not expect this when we started the order-selection check; our working assumption going in, informed by the two-seed version, was that it would turn out to be more noise. Six seeds said otherwise on two of four datasets.

Order selection’s benefit comes at a compute cost worth reporting. Fitting all four candidate orders instead of one produces a single-CPU wall-clock overhead (full pipeline, one seed) of $2.04\times$ on FD001, $3.11\times$ on FD002 (largest on the dataset where the benefit is also largest), $1.68\times$ on FD003, and $2.93\times$ on FD004. This is a one-time preprocessing cost, not a per-inference one. The hybrid architecture is also larger: concatenating trend features roughly doubles LSTM input width, raising parameter count by 7.2% on the 15-sensor datasets and 9.8% on the 21-sensor ones. A practitioner should weigh a roughly $3\times$ preprocessing cost and a sub-10% parameter increase against a gain that, per Table V, is real on two of four datasets and absent on the other two.

D. Window-Length Ablation

Table VI and Fig. 4 report a window-length ablation on FD001 ($W \in \{10, 20, 30, 50\}$), each trained with a shorter early-stopping patience than the main experiments for faster iteration across four window sizes. We initially ran this at three seeds (42, 123, 2024) and, consistent with the seed-sensitivity we found in Section V-A, extended it to the same six seeds used everywhere else in the paper before treating the pattern

TABLE VI

WINDOW-LENGTH ABLATION ON FD001, MEAN $\pm$ STD OVER 6 SEEDS (TEST RMSE, CYCLES).

W (cycles)	10	20	30	50
Plain	18.60$\pm$0.55	17.39 $\pm$ 0.41	14.69 $\pm$ 0.68	14.46$\pm$0.69
Hybrid	18.62 $\pm$ 0.65	16.71$\pm$0.29	14.15$\pm$0.70	14.63 $\pm$ 1.01

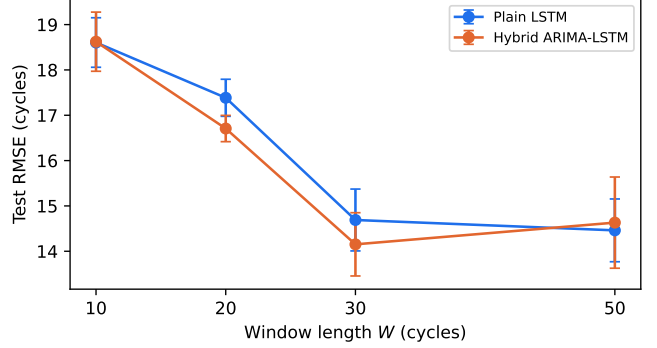
FD001: window length vs. RMSE, mean $\pm$ std over 6 seeds

Fig. 4. FD001 test RMSE vs. window length, mean $\pm$ std over six seeds. The hybrid model’s advantage is concentrated at $W=20$ and $W=30$ and is within noise at $W=10$ and $W=50$; not the clean peak-and-reverse pattern a three-seed average had suggested.

as reliable. The three additional seeds changed the picture. At three seeds, the hybrid model’s advantage appeared to grow monotonically with window length up to $W=30$ and then reverse at $W=50$. At six seeds, that monotonic story does not hold: $W=20$ and $W=30$ both show a real, consistent hybrid advantage (0.68 and 0.54 cycles), while $W=10$ and $W=50$ are both close to flat, with the sign of the small difference at each going in opposite directions across the two short-and-long extremes (-0.02 at $W=10$, -0.17 at $W=50$) rather than showing a clean reversal. The honest summary is a middle-window advantage bounded on both sides by noise, not a smooth peak-and-reverse curve.

VI. DISCUSSION

The headline finding is about evaluation, and it points to a real fix. We went into this project expecting a yes-or-no answer to whether a trend feature helps an LSTM predict RUL. Under a fixed ARIMA order, the specific answer moved as we checked our own work more carefully: at three seeds, FD001 looked like a clean, significant, explainable disadvantage for the hybrid model; at six seeds, that result was gone and FD004 had taken its place, a domain-specific instance of the pattern Henderson et al. [12] and Bouthillier et al. [14] document in reinforcement learning: a configuration can be exactly reproducible and still not representative of what the method does on average. We do not think the story ends there, though. Once we powered the order-selection check to the same six seeds, it stopped looking like a third source of noise and started looking like an explanation: on FD002, replacing the fixed order with a properly AIC-selected one

turns a nominal disadvantage into a significant, 6-of-6-seed advantage over the plain LSTM (Section V-C). On FD004, it erases the fixed-order disadvantage entirely. On FD001 and FD003, it does neither.

What we think is actually going on. An ARIMA(1,1,0) model, fit to a 30-cycle window, is a restrictive assumption, a single autoregressive lag on the first difference. Our order-selection results show that assumption was wrong for this data 75–83% of the time; (0, 1, 1), a moving-average model on the differenced series, fit better almost everywhere. A trend feature built from a poorly-fit model is not neutral, it is actively misleading, noise correlated with the raw signal it summarizes, and Section V-C’s fixed-vs-selected comparison suggests that noise was suppressing the hybrid model’s performance on FD002 and FD004. Once well-fit, on FD002 at least, it appears to supply real information the plain LSTM was not otherwise recovering, consistent with the window-length ablation (Section V-D), where the hybrid model’s advantage is concentrated in a middle range of window lengths. We ran three further checks aimed directly at this, and report what they actually showed rather than the cleaner story we were hoping for.

Is the order choice itself part of the story? No. The AIC-preferred order’s frequency was exactly stable across all six seeds within each dataset, 0.0 percentage-point standard deviation on every order (full breakdown in our public repository). The ARIMA step is fully deterministic given the data; whatever varies between seeds comes from the LSTM’s own training stochasticity, not order instability.

Is FD002’s win concentrated in one operating regime, or broad? Broad, with one exception. Stratified by FD002’s six operating regimes, the order-selected hybrid beats the plain LSTM in five of six (0.3–1.4 cycles lower MAE) and loses in one (regime 5, by 1.1 cycles), an exception we cannot explain.

Does FD002 have more ARIMA-explainable sensor trends, which would give a direct mechanistic reason? Weakly, and not fully satisfyingly. Fitting ARIMA(0, 1, 1) to 25 training units per dataset and measuring the fraction of each differenced sensor’s variance the fitted trend explains ($R^2 = 1 - \text{Var}(\text{residual})/\text{Var}(\text{diff})$), FD002’s mean R^2 (0.454) is statistically higher than all three others under a Welch’s t -test (FD001: +0.021, $p < 0.001$; FD003: +0.041, $p < 0.001$; FD004: +0.007, $p = 0.027$), but the gap against FD004 specifically, the dataset that matters most since it does not share FD002’s win, is tiny despite clearing $p < 0.05$. Statistically significant but practically negligible is not, in our view, an adequate mechanistic account.

If naive concatenation is the problem, does a learned gate fix it? No, not at the seed power that matters. The direct architectural fix for a poorly-fit trend feature misleading a fixed concatenation is to let the network learn how much to trust it. We built a gated-fusion variant, a per-timestep, per-channel sigmoid gate on the AIC-selected trend feature, trained under the same six-seed protocol on all four datasets (Table VII). A single-seed pilot (seed 42) looked promising, competitive with the selected-order hybrid on three of four datasets. At full six-seed power that promise did not hold: gated fusion beats the

TABLE VII
GATED-FUSION RMSE, MEAN $\pm$ STD OVER 6 SEEDS, VS. PRIOR MODELS.

Dataset	Plain	Fixed	Selected	Gated
FD001	14.43	14.92	14.38	14.97 $\pm$ 0.74
FD002	14.59	14.93	13.77	13.93 $\pm$ 0.35
FD003	14.72	14.87	14.45	14.33$\pm$0.90
FD004	14.92	15.71	14.90	15.03 $\pm$ 0.47

selected-order hybrid’s RMSE only on FD003, and a paired Wilcoxon test finds no significant difference from either the plain LSTM or the selected-order hybrid on any dataset (all $p > 0.13$). This is the second time a single-seed result looked like a broad win and six seeds showed otherwise (Section V-A); a null result for the fix our own Discussion proposed is worth stating plainly.

Taken together, these four checks rule out order-choice instability, confirm the FD002 effect is broad rather than regime-concentrated, leave the most direct mechanistic hypothesis we tested, differential trend linearity, only weakly supported, and show that the one architectural fix our own diagnosis suggested does not produce a broad win either. We think this is a more honest place to leave the question than forcing a mechanism onto data that does not clearly support one.

FD003’s variance is worth a separate note. The plain LSTM’s across-seed std on FD003 (2.06 RMSE) is the largest of any model-dataset pair in Table III. We have no confident explanation and would rather flag it as unexplained than invent one.

What we would tell someone deciding whether to add this kind of feature. Do not treat a single run, or even three seeds, as sufficient evidence either way. Our own answer changed twice before it stabilized: a nominal loss, then a statistically framed loss that itself moved between two sets of three seeds, and only at full six-seed power did a consistent picture emerge, that a naively fixed ARIMA order actively hurts this architecture, and that fixing the order, not abandoning the idea, is where the real effect was hiding.

Concretely, three checks are cheap enough to be routine before reporting a trend-augmented RUL model as an improvement or a failure: rerun under at least five or six seeds, not one to three (Section V-A); apply a multiple-comparison correction whenever the test repeats across datasets (Section V-B); and treat a fixed ARIMA order as a choice to be checked, not assumed (Section V-C), where two of our four datasets’ real result was hiding.

VII. LIMITATIONS AND THREATS TO VALIDITY

The window-length ablation (Section V-D) uses six seeds on one dataset (FD001) and a shorter training budget than the main experiments; it is a single-dataset finding we have not attempted to replicate on FD002–FD004, or under the AIC-selected order. Our comparison against Sharma [11] in Section V-A is informal, since we do not know whether that paper’s RUL cap, window length, or data split are close enough to ours to call it a controlled comparison. We tried four ARIMA

orders, one recurrent architecture, and two fusion mechanisms (concatenation and gating, Section VI), but not an auxiliary trend-prediction loss or a sweep of LSTM hidden size or depth, any of which could plausibly interact with Section V-C’s effect. We do not have an explanation for FD003’s unusually high plain-model variance across seeds, and we are reporting it as an open observation rather than resolving it. We investigated why order selection produces a significant, seed-robust win specifically on FD002 (Section VI) and came away with only a weak, statistically-significant-but-practically-tiny signal on one candidate mechanism (differential trend linearity vs. FD004) and no explanation at all for why regime 5 within FD002 is the one place the win does not hold; a fuller account would likely need per-sensor rather than per-dataset analysis, which we did not attempt. Finally, every experiment here uses NASA’s simulated turbofan data, and whether any of these patterns extend to real industrial IoT sensor streams with different noise and missingness characteristics is a question we are not in a position to answer.

VIII. CONCLUSION

We set out to test whether giving an LSTM an explicit per-sensor trend feature, computed with an ARIMA model, improves remaining useful life prediction on the NASA C-MAPSS benchmark. Under the fixed ARIMA order common in prior work, averaged over six seeds across all four sub-datasets, it does not: the hybrid model’s mean RMSE is at or above a plain LSTM’s everywhere, significantly so on FD004. Under a per-channel, AIC-selected order, tested at the same six-seed power, the picture changes: the selected-order hybrid significantly and consistently beats the fixed-order hybrid on two of four datasets, and on FD002 it significantly and consistently beats the plain LSTM baseline itself, the one result in this paper we consider a genuine, seed-robust architectural win rather than a null result or an artifact of insufficient replication. A CNN-LSTM baseline with no ARIMA feature at all shows the same dataset-dependent pattern rather than a uniform result, significantly worse on FD002 and non-significantly best-in-table on FD003, which we take as evidence that FD002’s resistance to added capacity and FD002’s response to order selection are both real and worth the compute cost of checking rather than assuming either. Getting to the order-selection answer took more checking than we originally planned, an early, thinly-seeded version looked unstable and nearly convinced us to report it as noise, and only full six-seed power revealed it was not. We think the practical lesson generalizes past our own experiment: on a benchmark with 100–260 test engines, a single training run, or a small handful of them, is not enough to support a confident claim in either direction about whether an architectural change like this one helps, and we would encourage some caution toward papers in this space, including the two we cite favorably in Section II, that do not report how many seeds their headline numbers rest on. A gated-fusion variant, our direct attempt at fixing naive concatenation with a learned per-channel gate, did not produce a broad win either at the same six-seed power

(Section VI). Future work includes a mechanistic account of why FD002 specifically benefits from a well-fit trend feature, an auxiliary trend-prediction loss as an untested alternative to gating, and validating the approach on proprietary industrial IoT sensor data.

CODE AND DATA AVAILABILITY

All code, data, and per-seed results underlying every table and figure here (120 training runs: 24 main, 24 order-selection, 24 ablation, 24 CNN-LSTM, 24 gated-fusion) are publicly available at <https://github.com/Pradeepsomasundaram/hybrid-arima-lstm-rul-cmapss>.

ACKNOWLEDGMENT

Portions of the experimental code and manuscript drafting were produced with the assistance of a generative AI system, under the direct supervision and review of the author, who verified all reported results, code, and claims. All experiments used the publicly available NASA C-MAPSS dataset with no fabricated or synthetic result values.

REFERENCES

- [1] A. Saxena, K. Goebel, D. Simon, and N. Eklund, “Damage propagation modeling for aircraft engine run-to-failure simulation,” in *Proc. Int. Conf. Prognostics and Health Management (PHM)*, 2008, pp. 1–9.
- [2] E. Ramasso and A. Saxena, “Performance benchmarking and analysis of prognostic methods for CMAPSS datasets,” *Int. J. Prognostics and Health Management*, vol. 5, no. 2, pp. 1–15, 2014.
- [3] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [4] X. Li, Q. Ding, and J.-Q. Sun, “Remaining useful life estimation in prognostics using deep convolution neural networks,” *Reliability Engineering & System Safety*, vol. 172, pp. 1–11, 2018.
- [5] Muthukumar G. and J. Philip, “CNN-LSTM hybrid deep learning model for remaining useful life estimation,” *arXiv:2412.15998*, Dec. 2024.
- [6] R. H. Chowdhury, N. Daiyan, F. Ahmed, M. R. Iqbal, and M. Sheikh, “Bi-cLSTM: residual-corrected bidirectional LSTM for aero-engine RUL estimation,” *arXiv:2603.00745*, Mar. 2026.
- [7] Y. Su and X. Fang, “A multi-head attention fusion network for industrial prognostics under discrete operational conditions,” *arXiv:2604.10248*, Apr. 2026.
- [8] S. Nie, Q. Liu, H. Ji, R. Hong, and S. Nie, “Integration of ARIMA and LSTM models for remaining useful life prediction of a water hydraulic high-speed on/off valve,” *Applied Sciences*, vol. 12, no. 16, art. 8071, 2022.
- [9] Y. Liu, H. Zhang, and L. Wang, “ARIMA and attention-based CNN-LSTM hybrid neural network for battery life estimation,” in *Proc. 2024 8th Int. Conf. Machine Learning and Soft Computing (ICMLSC)*, 2024, doi: 10.1145/3647750.3647759.
- [10] U. Khan, D. Cheng, F. Setti, F. Fummi, M. Cristani, and L. Capogrosso, “A comprehensive survey on deep learning-based predictive maintenance,” *ACM Trans. Embedded Computing Systems*, vol. 25, no. 2, Apr. 2025.
- [11] K. Sharma, “Uncertainty-aware deep learning framework for remaining useful life prediction in turbofan engines with learned aleatoric uncertainty,” *arXiv:2511.19124*, Nov. 2025.
- [12] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in *Proc. AAAI Conf. Artificial Intelligence*, 2018, pp. 3207–3214.
- [13] Z. C. Lipton and J. Steinhardt, “Troubling trends in machine learning scholarship,” *arXiv:1807.03341*, Jul. 2018.
- [14] X. Bouthillier, C. Laurent, and P. Vincent, “Unreproducible research is reproducible,” in *Proc. 36th Int. Conf. Machine Learning (ICML)*, 2019, pp. 725–734.
- [15] A. Goel, S. Galchar, and S. Kanu, “Remaining useful life estimation for turbofan engines: a comparative study of classical, CNN, and LSTM approaches,” *arXiv:2604.27234*, Apr. 2026.